Formal Verification of Lightning-Network Multi-Hop HTLC Routing in Tamarin 1.8

A modular, machine-checked model with security analysis and an empirical study of the prover's tractability boundaries

(author)

July 15, 2026

Abstract

We formalise Lightning-Network multi-hop HTLC (Hash Time-Locked Contract) routing in the Tamarin symbolic prover (v1.8) under a full Dolev–Yao adversary. The protocol is captured as a *modular package of five theories* totalling **58 machine-checked lemmas**, covering the two-party channel lifecycle, multi-hop routing with fees, staggered-CLTV timing safety, and the necessity of the underlying liveness assumption. Beyond re-establishing the standard safety properties (payment atomicity, intermediary safety, value conservation), we (i) harden the model against a forwarding-replay flaw by replacing a global axiom with a structural linear-resource guard; (ii) machine-check the known *wormhole* fee-theft attack, both as a reachability result and quantified to the exact stolen fees; (iii) prove economic-soundness theorems (end-to-end conservation, strictly positive fees, no value creation) and expose a griefing DoS; and (iv) establish *assumption minimality* empirically—removing the intermediary-claim restriction yields a concrete counterexample. A recurring theme is *tractability*: we pin down the precise combination of ingredients (signing + natural-number induction + an unbounded block clock) that exhausts the prover, and map two further boundaries—signed-message N-hop routing and reusable channels—where automated search stops scaling. Every positive and negative result was executed, not assumed.

Contents

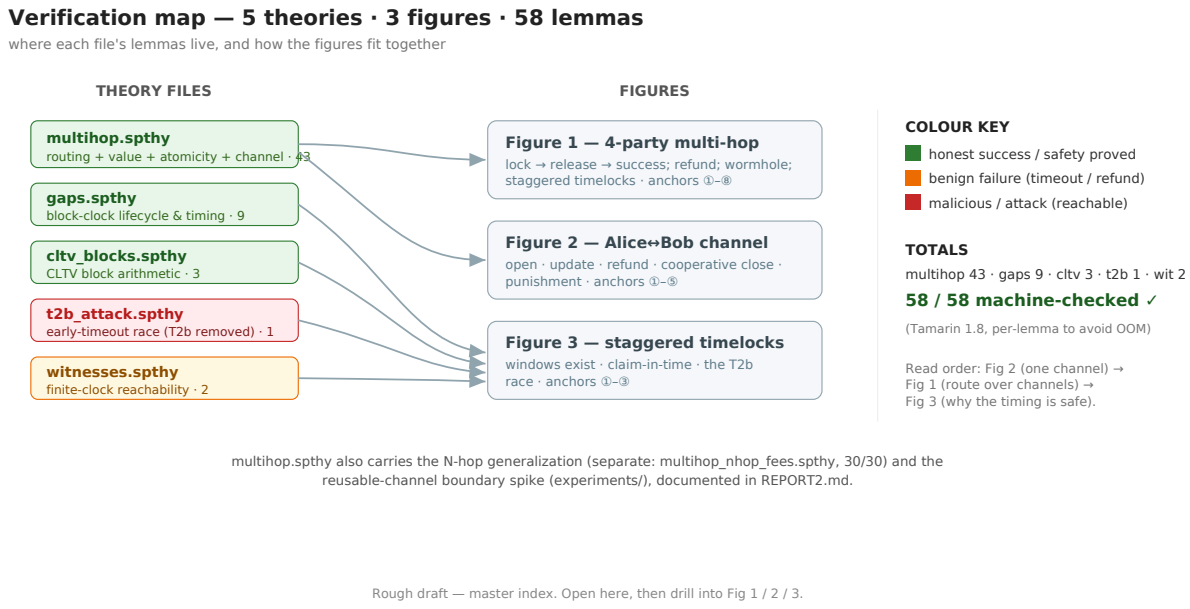

Figure 1: Verification map: five theory files, three figures, 58 lemmas. Green = honest success / safety proved; amber = benign failure (timeout / refund); red = malicious / attack shown reachable.

1 Introduction and scope

Lightning is a Layer-2 payment-channel network: parties transact off-chain and use the blockchain only to settle disputes. A multi-hop payment routes value along a path Alice $\to$ Bob $\to$ Carol $\to$ Dave by chaining HTLCs that share one payment hash $y = H(x)$; the receiver reveals the preimage x, which propagates back along the path, redeeming each HTLC. Our reference model is the standard textbook presentation (Maffei, TU Wien, *Cryptocurrencies*, Lecture 11): a four-party path, per-hop fees, staggered CLTV timelocks $3t/2t/t$, and a two-round "lock then release" commit.

Scope. We model integrity and safety under a Dolev–Yao adversary who controls the network and may compromise long-term keys. We *do not* model payment privacy / unlinkability (a confidentiality property requiring onion routing and observational-equivalence proofs)—this is stated as an explicit exclusion, not silently omitted. Timing is treated relationally (orderings of timeout events) except where a concrete block-height model is used to *derive* the CLTV windows.

2 Architecture: a modular five-theory package

A single monolithic theory is intractable: combining Tamarin's signing equational theory, natural-number induction, and an unbounded block clock exhausts memory. Crucially this was *tested*, not assumed—any *two* of the three ingredients remain tractable; only the full triple OOM-kills the prover. The model is therefore split so that no single theory carries all three (Table 1).

File	Aspect	Lemmas	Builtins
`multihop.spthy`	lifecycle + routing + value/fees + atomicity	43	hashing, signing, nat
`gaps.spthy`	block-clock lifecycle & CLTV timing safety	9	natural-numbers
`cltv_blocks.spthy`	pure CLTV block arithmetic	3	natural-numbers
`t2b_attack.spthy`	early-timeout race (liveness assumption removed)	1	—
`witnesses.spthy`	finite-clock reachability witnesses	2	natural-numbers
Total		**58**	

Table 1: The five theories. `multihop.spthy` combines signing + nat once the unbounded clock is dropped and timing is kept relational—which is why value/fee conservation could be merged into it.

3 Methodology

All results were produced under a strict *test-first* discipline: a property is claimed only after Tamarin reports it, and any "plausible" existence result is re-checked with a tightened variant before being trusted (Section 7 shows this catching a false positive). Restrictions (global trace filters) are treated as *assumptions*, not proofs; a suspiciously short proof is a warning sign, and each restriction is either derived elsewhere or documented as a liveness boundary.

Per-lemma invocation (rather than one bulk `-prove`) is used on the large theory to avoid OOM:

```
LANG=C.utf8 LC_ALL=C.utf8 tamarin-prover multihop.spthy \
  --heuristic=c --stop-on-trace=seqdfs --derivcheck-timeout=0 --prove=<Lemma>
```

The flag `-stop-on-trace=seqdfs` rescues the heavy existence witnesses but *hangs* the natural-number arithmetic theories, so it is applied per-file. The whole suite is driven by `run.py` (or the `Makefile`, which delegates to it).

4 Results: the core model

4.1 Multi-hop routing (`multihop.spthy`)

Figure 2 maps the routing, value, and attack lemmas onto the four-party flow; Figure 3 maps the channel-lifecycle lemmas onto a single Alice↔Bob channel. All 43 lemmas verify.

4.2 Timing safety (`cltv_blocks`, `gaps`, `t2b_attack`)

Figure 4 shows the staggered-timelock story. `cltv_blocks` *derives* the CLTV-delta invariant $d_{\text{out}} < d_{\text{in}}$ from concrete block numbers—i.e. the claim windows provably exist; `gaps` adds block-clock lifecycle safety (no HTLC after close, outcome exclusivity, redeem/refund reachability); and `t2b_attack` exhibits the early-timeout race that reappears once the liveness assumption T2b is removed.

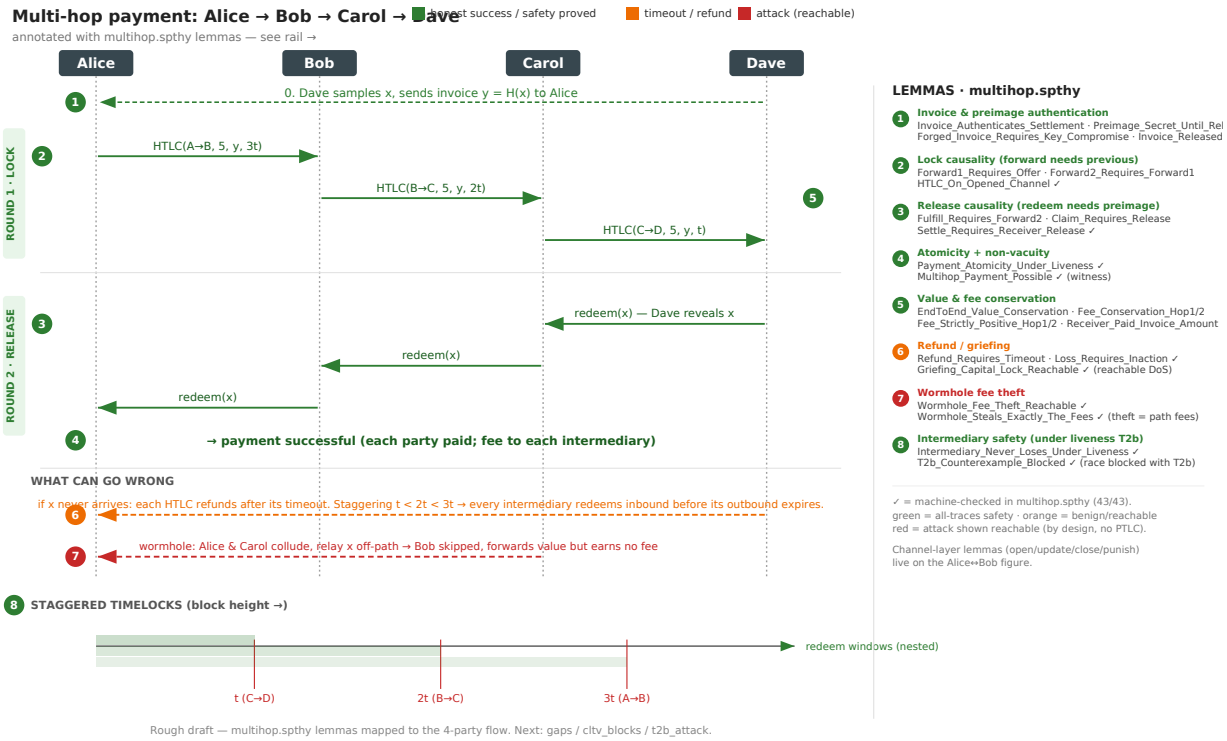

Figure 2: Four-party multi-hop payment. Round 1 locks HTLCs forward with staggered timelocks $3t/2t/t$; Round 2 releases the preimage backward. Numbered anchors 1–8 key to the `multihop.spthy` lemma rail. The red line is the wormhole; the amber line the timeout/refund unwind.

4.3 Statistics

5 Security analysis

5.1 Forwarding-replay hardening (axiom → structure)

The forwarding rules read the outgoing amount/fee from the network and are gated only by a public HTLC-offer message. A Dolev–Yao replay of that offer could re-fire an honest forward, locking a *second* HTLC on the same channel output with an adversary-chosen amount. This was found empirically via an audit probe (double-forward: reachable) and initially blocked with a global axiom `OneHTLCPerPtr`. We then replaced the axiom with a *structural* guard: each channel open emits one linear `Free(ptr)` token, and every HTLC-add consumes it. "At most one HTLC per output" is now a consequence of linear-fact consumption—one fewer assumption for the same guarantee. After the change the double-forward probe is falsified structurally (100 steps, no axiom) and all 43 lemmas still verify. We also confirmed that a symmetric "double-redeem" is *not* a vulnerability: the redeem/settle rules each consume a linear pending fact, so replay cannot double-spend an output.

5.2 Wormhole attack (reachable and quantified)

The wormhole attack (Malavolta et al., NDSS'19): two colluding path members relay the preimage out-of-band, skipping an honest intermediary who forwards value but earns no fee. In the symbolic

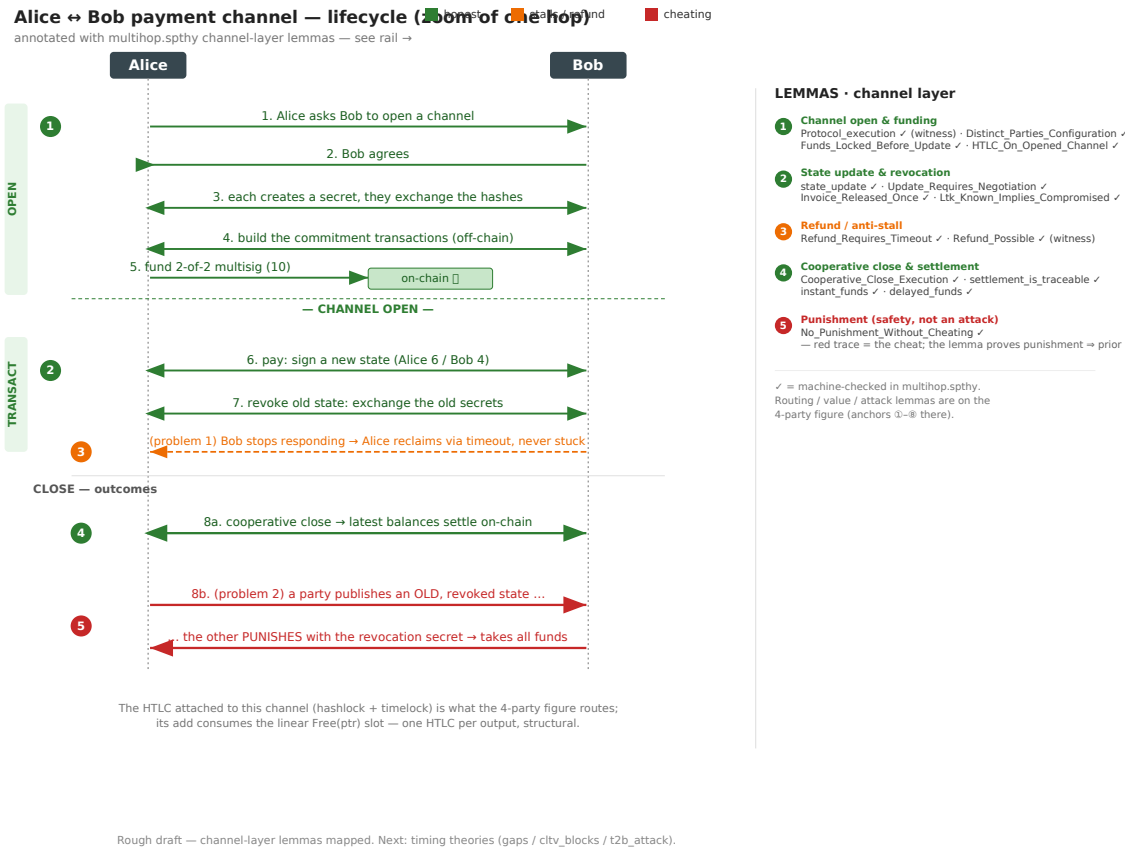

Figure 3: The Alice↔Bob channel lifecycle (a zoom of one hop): open, state update, refund, cooperative close, and punishment. Note anchor 5: the red trace is the *cheat*, but the lemma `No_Punishment_Without_Cheating` is a safety guarantee (punishment ⇒ prior cheat).

model the Dolev–Yao network *is* the colluders' side channel. We machine-check it twice: `Wormhole_Fee_Theft_Reachable` (51 steps) exhibits a trace where the payment settles yet the middle hop is never redeemed and no intermediary earns a fee, with *no* key compromise; and `Wormhole_Steals_Exactly_The_Fees` (81 steps) quantifies the theft to precisely the path fees $\%f_1\% + \%f_2$, tying the attack to the value layer. This is the known limitation of the base HTLC construction (the fix in the literature is anonymous multi-hop locks / PTLCs, out of scope here); making it an explicit verified result turns "we did not model it" into "we machine-checked that the model exhibits it, and why."

5.3 Economic soundness and griefing

Three all-traces theorems, none of which is assumed anywhere: `EndToEnd_Value_Conservation` (the sender locks the receiver's amount plus a strictly positive total fee; no value created or destroyed); `Fee_Strictly_Positive_Hop1/2` (every hop's fee is > 0—structural, because the natural-number sort has no $\%0$, so a "free forward" is unreachable); and, as a reachability, `Griefing_Capital_Lock_Reachable` (an honest intermediary induced to lock capital then refund on timeout, with no payment completing—the standard PCN griefing DoS).

Group	Class	Representative lemmas
Invoice / preimage auth	■	`Invoice_Authenticates_Settlement`, `Preimage_Secret_Until_Released`, `Forged_Invoice_Requires_Key_Compromise`
Lock causality	■	`Forward1_Requires_Offer`, `Forward2_Requires_Forward1`, `HTLC_On_Opened_Channel`
Release causality	■	`Fulfill_Requires_Forward2`, `Claim_Requires_Release`, `Settle_Requires_Receiver_Release`
Atomicity / non-vacuity	■	`Payment_Atomicity_Under_Liveness`, `Multihop_Payment_Possible`
Value & fees	■	`EndToEnd_Value_Conservation`, `Fee_Conservation_Hop1/2`, `Fee_Strictly_Positive_Hop1/2`, `Receiver_Paid_Invoice_Amount`
Refund / griefing	■	`Refund_Requires_Timeout`, `Loss_Requires_Inaction`, `Griefing_Capital_Lock_Reachable`
Wormhole	■	`Wormhole_Fee_Theft_Reachable`, `Wormhole_Steals_Exactly_The_Fees`
Intermediary safety	■	`Intermediary_Never_Loses_Under_Liveness`, `T2b_Counterexample_Blocked`
Channel lifecycle	■	`state_update`, `Protocol_execution`, `Cooperative_Close_Execution`, `No_Punishment_Without_Cheating`, `Funds_Locked_Before_Update`

Table 2: Lemma groups in `multihop.spthy` (43 total). Class: ■ safety proved, ■ benign/reachable, ■ attack reachable.

5.4 Assumption minimality (T3 is load-bearing)

A natural question is whether the intermediary-claim restriction T3 is redundant given T1 (redeem-before-timeout) and T2b (honest parties act before the inbound timeout). We tested it by removing T3 and re-running the suite: the safety lemma `Intermediary_Never_Loses_Under_Liveness` is then *falsified*, Tamarin finding a concrete 59-step counterexample. T2b orders the two timeouts but does not force the node to *sweep* the incoming HTLC once its outgoing one is redeemed; that action is exactly what T3 encodes. Hence the three timing restrictions are minimal: none is implied by the others.

6 Generalisation to N hops

The core model fixes the path at three hops. Replacing the two hardcoded forward rules with one *generic* rule was attempted in several encodings, all run:

The linear-fact route yields `multihop_nhop.spthy` (28/29 lemmas; the one open lemma, `Settle_Requires_Receiver_Release`, keys on the terminal settle and does not terminate) and, with amounts/fees added, `multihop_nhop_fees.spthy` (30/30). The framing is a sound proof layering: concrete model with full crypto adversary at three hops; abstraction with idealised channels for routing safety at arbitrary N.

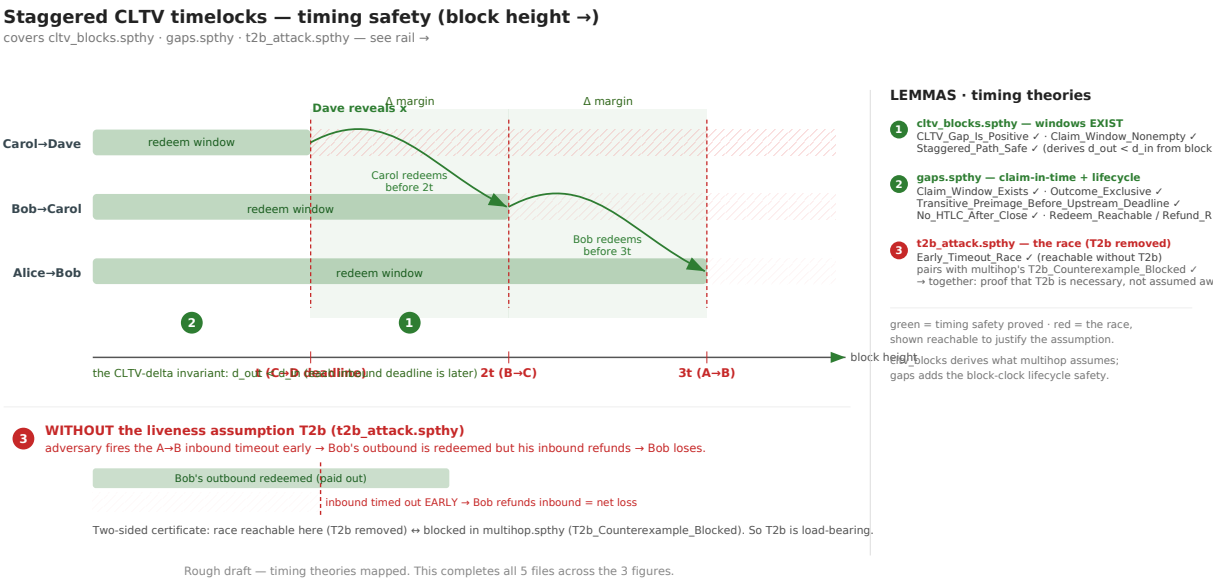

Figure 4: Staggered CLTV timelocks. The preimage propagates backward with a Δ margin at each hop because every inbound deadline is later than its outbound one. Bottom (red): with the liveness assumption T2b removed, the adversary fires the inbound timeout early and the honest intermediary suffers a net loss—`Early_Timeout_Race`. This pairs with `T2b_Counterexample_Blocked` in `multihop.spthy`, forming a two-sided certificate that T2b is necessary.

7 What failed, and the tractability boundaries

We record negative results explicitly—they are the paper's methodological contribution as much as the positive ones.

- **The three-ingredient OOM.** Signing + natural-number induction + an unbounded block clock together exhaust the prover; any two are fine. This is the reason for the modular split (Section 2).
- **Signed-message N-hop routing** (Table 4): origin analysis on the self-referential `'htlc'` term does not terminate, even for reachability, even with a fresh id.
- **A rejected restriction.** An initial fix `OneRedeemPerPtr` ("at most one redeem per output") was tested and *rejected*: it falsified the honest witness `Multihop_Payment_Possible`, because the two `Redeemed` events on the sender's output in an honest run are the two endpoints' views of one hop, not two spends. The correct guard was `OneHTLCPerPtr` (later made structural).
- **A caught false positive.** An exists-trace probe that shared only the payment *hash* between an offer and a payment appeared to show "value creation" (receiver paid more than the sender locked, no compromise). Tightening it to a single *linked* payment chain *falsified* it, while a control confirmed the chain is otherwise reachable—so the apparent attack was invoice-hash reuse across two unrelated offers, not a soundness bug. This is the adversarial-verification discipline in action.
- **Reusable channels (boundary).** We staged a refactor toward faithful reusable channels: Stage 1 (structural single-HTLC-per-output via the `Free(ptr)` slot) ships and re-verifies

Lemma (representative)	Steps	Time	Kind
`Multihop_Payment_Possible`	70	∼77 s	exists-trace (heaviest)
`Forward2_Requires_Forward1`	274	∼47 s	all-traces
`Forward1_Requires_Offer`	266	∼46 s	all-traces
`Wormhole_Steals_Exactly_The_Fees`	81	∼45 s	exists-trace
`Wormhole_Fee_Theft_Reachable`	51	∼40 s	exists-trace
`Intermediary_Never_Loses_Under_Liveness`	2	<1 s	all-traces (under T2b)
`EndToEnd_Value_Conservation`	2	<1 s	all-traces

Table 3: Representative per-lemma statistics for `multihop.spthy` (43 lemmas verify per-lemma sequentially in ≈16 min total on the test machine; average ≈26 s; many trivial lemmas finish in <1 s).

Encoding	Outcome	
Signed-message generic forward	self-referential signed term; origin analysis recurses, even a one-hop witness OOMs	■
+ fresh per-hop id	fixes replay but *not* termination—still a signed term producible only by a forward	■
Linear-fact routing (idealised channel)	safety invariants verify by induction over the chain, any N	■

Table 4: N-hop encodings. The bounding factor is cryptographic origin-analysis recursion on self-referential signed messages, not path length.

43/43. Stage 2 (return `Free(ptr)` on resolution, so a channel carries sequential HTLCs) was spiked and is a clean tractability boundary: without `-auto-sources` every lemma returns "analysis incomplete" (the reuse loop leaves unbounded partial deconstructions); *with* `-auto-sources` the backward-reasoning safety lemmas verify (∼45 s each), but every exists-trace witness and the interval lemma `No_Concurrent_HTLC_Per_Output` fail to converge (killed 500–700 s). Reusable channels are expressible and their core safety survives, but the loop reintroduces the witness/liveness non-termination the modular split avoids; Stage 1 is therefore the shipped model.

8 Reproduction

```
export LANG=C.utf8 LC_ALL=C.utf8
python3 run.py # full core suite (5 theories)
python3 run.py multihop.spthy # one theory
make nhop # the N-hop extension (30/30)
```

Requires Tamarin 1.8 and Maude 3.1. The Makefile delegates to `run.py`, the single source of truth for per-file flags.

9 Conclusion

We delivered a modular, fully machine-checked model of Lightning multi-hop HTLC routing (58 lemmas over five theories), hardened it against a forwarding-replay flaw by turning a global axiom into a structural invariant, machine-checked the wormhole attack (reachable and quantified) and a

griefing DoS, proved economic-soundness theorems, and established the minimality of the timing assumptions empirically. Equally, we mapped where the prover stops scaling: the three-ingredient OOM, signed-message N-hop routing, and reusable channels. The protocol-level results confirm expectations; the durable contribution is methodological—what it takes to keep Tamarin tractable on a stateful, time-locked, arithmetic protocol, with Lightning as the validating case study, and a discipline in which every claim (positive or negative) was executed rather than asserted.

A Figure index

Figure 1 is the master map. Figures 2 and 3 carry `multihop.spthy`; Figure 4 carries the three timing theories. All four are provided as PDF (embedded) and SVG (source) under `paper/figures/`. Colour key throughout: ■ honest/safety-proved, ■ benign failure, ■ attack reachable.